

Hello!

This is the explanation of every thing in the (resources/)aust/ folder!

This will outline the different way of declaring JSON for the engine to properly read it. As well as mandatory declarations, without which your mod might not work! It also contains examples on full paths from the base game.

Anything in green is considered a comment and shouldn't be put within any of the files you write.

I will try to inline most declarations here, but some are just huge / have multiple ways of being declared, so you might have to look at the JSON provided in the same folder.

At the bottom of the document, you can find miscellaneous stuff, such as events, how to declare time, etc.

Improper declaration could result, in the best case, in the UI / maps / sprites / etc not loading partially or fully, and in the worst case, crash the application.

- **LICENSE.txt**

License for the assets used and code. Code is MIT, meanwhile custom game assets and the maps are copyrighted. Third party assets are listed.

- **mod.json**

Has 2 mandatory declaration. Points to the configuration file. Anything else up to the maker.

```
{  
    "configFile": <path>.json, // It should be the path to the 'configuration.json' from  
    where the mod.json is located; after being expanded it will be aust/<path>.json  
    "name": <name> // Used by the ModManager to display that the mod has been  
    found in console.  
}
```

- **scores/**

The scores folder. Entirely optional, I just preferred to have it. Nothing points to it, but can be referenced in „**configuration.json**“.

- **scores/scores.json**

This should follow the same declaration type as in the base game's file. Does not point to anything. Pointed to by „**configuration.json**“

```
[
```

```

    { "name": "PlayerOne", "caught": 2, "time": 145 },
    { "name": "PlayerTwo", "caught": 0, "time": 115 },
    ... // And so on
]

```

- **saves/**

The saves folder. Here is where the saves are located. Pointed to by „**configuration.json**”

- **saves/save[x].json**

This is where (some incomplete) magic of the engine is saved. It usually works (usually).

See [saveExplained.json](#)

- **resources/**

In this folder, every resource, from UI, to maps, to dialogue, lives. Nothing points to it implicitly, but can be referenced in „**configuration.json**”.

- **resources/ui.json**

Should follow the same declarations. The paths are to point to files WITHIN the „**ui/**” directory. Pointed to by „**configuration.json**”

```

{
    "mainMenu": "mainMenu.json", // Once parsed fully, this would be aust/...
    /ui/mainMenu.json
    "pauseMenu": "pauseMenu.json",
    "dialogue": "dialogue.json"
}

```

- **resources/tiles.json**

Contains the declaration of tiles. A few of the tiles must have a given name, while the rest can have any name. Pointed to by „**configuration.json**”. The values of source refer to the position on the spritesheet. Also used in the dialogue file to get the up and down arrows for the dialogue box.

Tiles in AUST aren't really 'tiles', or well they are, but they can be multiple tiles in a single tile. I have no idea what to really call these 'multiple-tile tiles'.

Due to engine limitations (and laziness until I get around to make it), here are the tiles whose names MUST be used for the engine to properly put them in their required places: PlayerUp, PlayerDown, PlayerRight, PlayerLeft, GuardLeft, GuardRight, GuardUp,

GuardDown, Chaselcon, SearchIcon. The game won't crash without them, but the entities will be invisible.

Placement types:

- „replace” (removes all other tiles on the spot, and then adds the tile).
- „add” (adds ontop of any existing tiles)
- „remove” (removes the tile(s), used in setTile)

Render layers (from lowest to highest priority; higher priority -> closer to the screen)

- „ground”
- „decoration”
- „actor”
- „foreground”
- „overlay”

Animation / sound types: „idle”, „steppedOn”, „lookAt”, „lookAway”, „walk”, „attack”, „death”

See [tilesExample.json](#)

• **resources/systemValues.json**

The system values for the app, includes the saved audio, dimensions of the screen and the isFullscreen flag, but unfortunately, only the saved audio value is used.

```
{
  "appDimensions": { "h": 1009, "w": 1920 },
  "isFullscreen": false,
  "masterVolume": 1.0
}
```

• **resources/settings.json**

Should follow the same declaration type; If missing or incomplete flags, they have default values when being initialised. Does not point to anything. Pointed to by „**configuration.json**”

• **resources/fonts.json**

Should follow the same declaration type. The name should point to a file in the „**fonts/**” folder. Pointed to by „**configuration.json**”

```
{
  „name”: „<name>.ttf”, // Once parsed, it will be aust/.../fonts/<name>.ttf
  ... // And so on
}
```

- **resources/flags.json**

Should follow the same declaration type. Does not point to anything. Pointed to by „**configuration.json**“. Note that you cannot yet add flags at runtime.

```
[  
    { "name": „<flagName>“, "initial": true OR false }  
    ... // And so on  
]
```

- **resources/configuration.json**

The big file that points to everything and has everything in it! Pointed to by the “**mod.json**” file. Points to almost every other file, directory and resource.

[See configurationExplained.json](#)

- **resources/audio.json**

Should follow the same declaration type, with the following mandatory. If there any sound declarations, they will point to the “**audio/**” folder. Pointed to by „**configuration.json**”

[See audioExample.json](#)

- **resources/ui**

The UI folder, this is where all the UI JSON files are located. Pointed to by „**configuration.json**“. Used whenever „**ui.json**” or cutscenes are queried.

- **resources/ui/[file].json**

The UI I spent quite some time making it load from a JSON (making stuff like the audio slider and fullscreen button work was terrible). Anyway, UI is pretty simple to make, so simple that in fact, cutscenes are actually UI as well! That’s why there’s no dedicated custscene example JSON.

[See uiExample.json](#)

- **resources/textures**

A general folder where you can put anything (but hopefully images). Nothing points to it implicitly, but can be referenced within the „**configuration.json**” file

- **resources/sprite_sheet.png**

The sprite sheet of the game. Pointed to by „**configuration.json**”. Used when the game loads the tiles.

- **resources/presets**

Folder where the <presetName>.json are located. Pointed to by „**configuration.json**“. Used whenever a preset is to be loaded.

- **resources/presets/[preset].json**

A preset for the map. A preset is a collection of tiles (arranged as matrix or vector), which the map can load. This makes copy pasting the same house easier! You can use 'skip', 'X' or null to skip tiles. Only referenced by name ([presetName])

```
[  
    ["skip", "Soil", "Forest", null],  
    ["Forest", "Water", "Water", "Soil"],  
    ["Soil", "Water", "Water", "Forest"],  
    ["X", "Soil", "Soil", "X"]  
]
```

- **resources/maps/**

The maps folder is where every map is located. Pointed to by „**configuration.json**“. Used whenever a map is loaded by name.

- **resources/maps/[map].json**

Look! A map! This contains everything a map does, tiles, enemies, presets, triggers, you want it, it's all yours my friend. Pointed to by „**configuration.json**“ (the starting map), and whenever a map is loaded by name.

[See mapExplained.json](#)

- **resources/images/**

Images folder. Anything can go in here. The icon image is also found here. Pointed to by „**configuration.json**“. Used whenever the icon is loaded for the window.

- **resources/images/[iconName].extension**

The icon for the game window (in the taskbar, and in the upper left corner of the window). Pointed to by „**configuration.json**“ (within the window object; only the name). Used at the start of the engine to get the icon. For obvious reasons, this should be a small square (64x64 or 32x32).

- **resources/fonts**

The font folder. Pointed to by „**configuration.json**” and used whenever „**fonts.json**” is queried.

- **resources/dialogue**

The dialogue nodes and banks folder. Pointed to by „**configuration.json**”. Used whenever a dialogue references a node or a bank.

- **resources/dialogue/[bank].json**

A dialogue bank file. Loaded whenever referenced in a dialogue declaration or node. Not pointed to by anything. Should be a vector of strings. In the folder view, there is no difference between nodes and banks, but when banks are referenced, they have the extension attached (“bankFile”: “bank.json”), while nodes are only referenced by name.

```
[  
    "Hmm... alright. Move along.",  
    "Okay, carry on.",  
    "I'll verify that later. For now, proceed.",  
    "Fine, just don't cause trouble.",  
    "You're lucky I'm in a good mood."  
]
```

- **resources/dialogue/[node].json**

A dialogue node file. This contains the type of dialogue (narration, choice), and the afferent lines, questions of each. In the folder view, there is no difference between nodes and banks, but when banks are referenced, they have the extension attached (“bankFile”: “bank.json”), while nodes are only referenced by name.

See “[dialogueNodes.json](#)”

- **resources/cutscenes/**

This is where the cutscenes are. Pointed to by „**configuration.json**”. Used whenever a cutscene is queried to be played (by name).

- **resources/cutscenes/[cutsceneName].json**

A cutscene in this engine is an UI element (an UIPanel to be exact). It has the same way of declaring it as a panel, which has been detailed in the “**resources/ui/[ui].json**”. What (may) make cutscenes unique is the fact that they usually have a cutscene text writer in them.

- **resources/audio**

This is where the audio files are located. Pointed to by „**configuration.json**”.

- **resources/audio/.../[file].extension**

An audio file. Pointed to by „**audio.json**“, within a sound declaration.

(EVIL) MISC STUFF

- **Just a question of time**

All timers in AUST are in milliseconds. And to avoid confusion, here are the ways to declare time in a JSON. They are automatically converted to milliseconds if they are given in seconds.

“value”: <number> -> this is interpreted as seconds

“value”: { “s”: <number> } -> seconds

“value”: { “seconds”: <number> } -> seconds

“value”: { “ms”: <number> } -> milliseconds

- **How to declare points / sizes / rectangles / padding / colors / alignment / tile positions.**

I can 99% guarantee that the way of declaring the mentioned things with OBJECTS works. Arrays / numbers may crash the engine for certain things, mostly because testing takes a long time and I am already burnt out on this.

Points:

- “value”: { “x”: 10, “y”: 10 }
- “value”: [10, 10] // X then Y
- “value”: 10 // Creates a point with X = Y = 10

Sizes:

- “value”: { “w”: 10, “h”: 10 }
- “value”: { “width”: 10, “height”: 10 } // The “w” and “width” are interchangeable, so you can mix both in the same declaration; { “w”: 10, “height”: 10 }
- “value”: [10, 10] // Width then Height
- “value”: 10 // Creates a size with width = height = 10

Rectangles:

- “value”: { “x”: 10, “y”: 10, “w”: 10, “h”: 10 } // Once again, “w” / “h” and “width” / “height” are interchangeable
- “value”: [10, 10, 10, 10]

Padding:

- "value": { "x": 10, "y": 10 }
- "value": { "horizontal": 10, "vertical": 10 } // "x" / "y" and "horizontal" / "vertical" are interchangeable.
- "value": 10 // horiz = vert = 10

Colors:

- "value": { "r": 10, "g": 10, "b": 10, "a": 255 }
- "value": [10, 10, 10, 255]
- "value": "0A0A0Ah"
- "value": "0xFF" // Expands to 0xFFFFFF
- "value": "#0A0A0A"

Alignment:

- "value": { "horizontal": left / center / right, "vertical": top / middle / bottom }

Tile positions. These contain points, but also ranges!

- "value": { "x": 10, "y": 10 } -> Tile at [10, 10]
- "value": { "x": [10, 11], "y": 10 } -> Tile at [10, 10], [10, 11]
- "value": { "x": 10, "y": { "range": [10, 12] } } -> Tile at [10, 10], [11, 10], [12, 10]

• Defines, now in JSON!

Now you might wonder: "What?". Defines are made so you can parametrise some of the UI layout calculations. In the root object of the file, define

```
"@defines": { // Name must be @defines
  "_SLOT_HEIGHT": 110, // The names here are up to the maker.
  "_SLOT_PADDING": 10, // These must be numbers!
  "_STARTING_Y": 100
},
```

Don't worry, these are removed after parsing, so no extra data is kept in memory. If there is no @defines key, the entire string substitution parsing is skipped (even if set to parse).

Now to use them:

```
"bounds": {
  "x": 150,
  "y": "_STARTING_Y + 2 * _SLOT_PADDING + 2 * _SLOT_HEIGHT",
  "w": 500,
  "h": "_SLOT_HEIGHT" // Names must match!
},
```

Now you might see that there's mathematical operations. That is not standard JSON, but it works. Only supports flat operations (meaning no parentheses allowed), and only addition,

subtraction, multiplication, division. Follows normal operator priority order, from left to right.

- **Actions and events**

Actions and events are really easy to understand, so here's all of them, in a file, of course.

[See eventsExample.json](#)

- **logs/gameLogs/**

Although not in the aust/ folder, this folder contains the output logs, in text format, of the game. Can be disabled in "**settings.json**". Can safely be deleted, it will be recreated again if needed.

- **logs/memoryLogs/**

Although not in the aust/ folder, this folder contains the output log of the memory tracker. This is meant to be a developer tool, so if you're a normal user who doesn't code or anything, it does not contain anything useful. The same logs are also outputted in the console (information for nerds).

- **resources/mods/**

This is where any mods live! Put mods in their own folder within this folder (so something like mods/MyMod), and within that folder, the mod.json (so mods/MyMod/mod.json).

The first AUST version will most likely NOT ship with a in-app mod selection screen, so until then, to play a mod, do the following:

- Create a shortcut to the main executable

- Right click -> options -> Target -> at the end add " -mod [modFoldername] " (without quotation marks)

- Run the shortcut (double click)

So for a mod called "myMod", it would be " -mod myMod ".

For the nerds (Linux users... wait does this even compile on Linux?), run from the command line with same parameters.

This feature is untested, so it might explode the engine (sorry!)